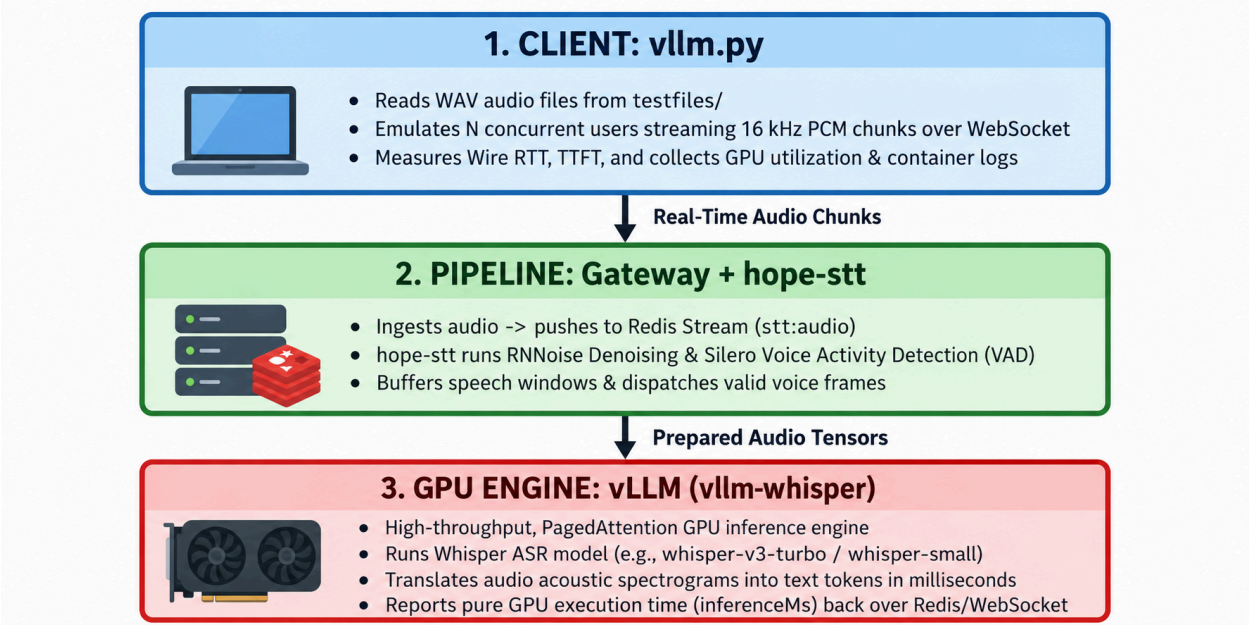
vLLM Real-Time Streaming Concurrency Load Testing Guide

This guide documents:

1. **What vLLM & `vllm.py` do** in this STT architecture.
2. **The exact 10-step execution lifecycle** when running `python3 vllm.py [args]`.
3. **How to run the tests** with various configurations and arguments.
4. **Generated outputs and performance artifacts**.

---

1. System Architecture & Component Roles



The Core GPU Engine (vLLM on Remote Host)

- **Engine**: vLLM is an inference engine optimized for fast throughput and low latency on NVIDIA GPUs.
- **Continuous Batching & PagedAttention**: Dynamically batches speech recognition queries across multiple concurrent streams, keeping inference latencies low.
- **Inference Execution**: Decodes acoustic spectrograms through Whisper neural weights into transcribed text tokens and returns execution telemetry.

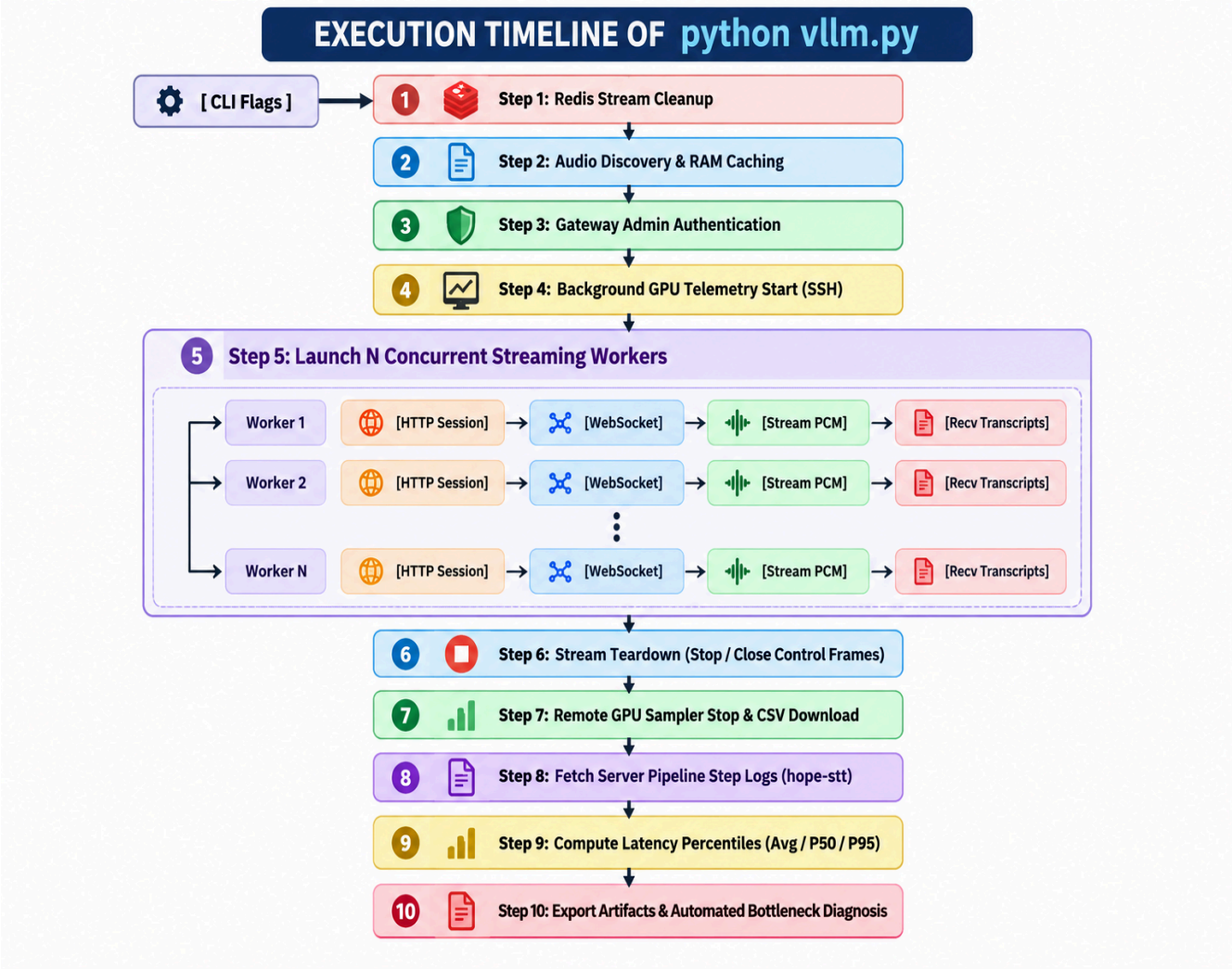
The Client Test Runner (`vllm.py`)

- **Zero Local ML Modeling**: The client machine only does network streaming and telemetry recording—no local ASR, VAD, or denoising.
- **Realistic Concurrency Simulation**: Simulates 1 to N simultaneous human speakers streaming audio over WebSockets at real-time pacing (`1.0×` speed).
- **End-to-End Latency Decomposition**: Separates network wire time, gateway proxy overhead, CPU preprocessing, and GPU decoding.

---

2. What `vllm.py` Does (10-Step Execution Lifecycle)

When you execute `python3 vllm.py [args]`, the script performs the following 10 steps in chronological order:



Step 1: Redis State Cleanup

- Connects to the Redis instance (`192.168.112.6:32522`).
- Deletes stale leftover streaming keys from previous runs (`stt:audio:*`, `stt:result:*`, `stt:control:*`, `stt:session:*`, `stream-session-meta:*`).
- **Purpose**: Prevents old unread audio or stale responses from contaminating current benchmark runs.

Step 2: Audio File Pool Discovery & RAM Pre-Caching

- Scans the audio directory (`testfiles/`) or reads specific files provided via `--audio`.
- Validates that files are playable 16 kHz 16-bit mono WAV.
- Loads all PCM bytes into memory upfront so local disk read speed does not bottleneck concurrent streaming.

Step 3: Gateway Authentication (HTTP REST)

- Sends an authentication request to `POST http://192.168.112.6:32458/api/v1/auth/login`.
- Retrieves and stores a valid JWT Bearer token for worker session creation.

Step 4: Background GPU Telemetry Start (via SSH)

- Connects via SSH to the remote GPU host (`nitik@192.168.112.2`).
- Starts a background sampling process running `nvidia-smi -lms 50` (sampling GPU utilization, VRAM usage, and power draw every 50 ms) writing to `/tmp/gpu_sample.csv`.

Step 5: Concurrent Streaming Execution (Per Worker)

For each concurrent worker (1 to N), `vllm.py` launches an asynchronous worker task:

1. **Startup Stagger**: Worker `i` sleeps for `(i - 1) × stagger_ms` (default 50 ms) to avoid overwhelming the gateway with a synchronized thundering herd.
2. **HTTP Session Creation**: Calls `POST /api/v1/audio/transcription-jobs/stream/session` to obtain a `sessionId`, an ephemeral WebSocket `ticket`, and worker capacity limits.

3. **WebSocket Handshake**: Connects to `ws://192.168.112.6:32458/ws/stt/stream?sessionId=...&ticket=...`.
4. **Server Handshake**: Awaits the server's `{"type": "ready"}` frame before sending audio data.
5. **Real-Time Streaming Loop**:
    - Slices the in-memory PCM audio into chunks (default 250 ms = 8,000 bytes).
    - Transmits frames over the WebSocket.
    - Paces chunks using a drift-free reference clock to maintain real-time human speaking cadence.
6. **Concurrent Transcript Receiver**:
    - Concurrently listens on the WebSocket for incoming transcript frames (`{"type": "transcript", ...}`).
    - Extracts server-reported GPU decode time: `inferenceMs`.
    - Measures Wire Round-Trip Time (WS RTT) from chunk transmission to transcript arrival.
    - Computes Gateway & Transit Overhead (`Wire RTT - inferenceMs`).

Step 6: Session Teardown & Drain

1. When audio completes, waits 1.0 second for socket buffers to drain into Redis.
2. Sends the control frame `{"type": "stop"}` to signal speech completion.
3. Awaits the server's finalization response (up to 180 seconds under heavy load).
4. Sends `{"type": "close"}` to cleanly close the session without triggering reconnection timers.
5. Issues an HTTP `DELETE /session/{sessionId}` to immediately release worker slots in `CapacityGuard`.

Step 7: Remote GPU Sampler Teardown & Retrieval

- Terminates the remote `nvidia-smi` background process on the GPU host.
- Downloads `/tmp/gpu_sample.csv` to `logs/concurrency_<N>/gpu_<N>.csv`.

Step 8: Granular Pipeline Step Extraction (Server Logs)

- Queries `docker logs hope-stt` on the GPU host over SSH.
- Parses timestamps from `stt.step.*` events matching active session IDs:
    - **Denoising time** (RNNoise)
    - **VAD time** (Silero VAD)
    - **Server Queue wait time** (Queue wait before GPU execution)
    - **Postprocessing & Redis publish time**

Step 9: Statistical Latency Calculations

Aggregates all worker metrics and calculates **Average**, **P50 (Median)**, **P95**, **Min**, and **Max** across all 10 stages of the pipeline.

Step 10: Artifact Generation & Bottleneck Diagnosis

Outputs the formatted summary table to stdout and writes complete audit logs into `logs/concurrency_<N>/`. Automatically diagnoses the primary bottleneck factor (Ingress, Queue Congestion, GPU Inference, or Egress).

---

3. How to Run `vllm.py`

Run all commands from the repository root:

1. Run Only Concurrency 32

Runs only the 32-concurrency test instead of the entire configured concurrency sweep:

python vllm.py --config benchmark_config.yaml --concurrency 32

2. Run a Custom Concurrency Subset with Custom Cooldown

Runs concurrency levels 4, 8, and 16 with a 10-second cooldown between runs:

python vllm.py --config benchmark_config.yaml --concurrencies 4,8,16 --cooldown 10

3. Test Burst Rate with 100 ms Chunks

Runs the test at burst speed (`0.0x` pacing) with 100 ms audio chunks:

python vllm.py --config benchmark_config.yaml --pace 0.0 --chunk-ms 100

4. Use a Specific Audio File

Uses a specific audio file instead of the `testfiles` directory:

python vllm.py --config benchmark_config.yaml --audio testfiles/gastro.wav

---

4. vLLM STT Streaming Load Test Configuration

The load test configuration is stored in `benchmark_config.yaml`.

Configuration Reference

Configuration	Default	Description
`concurrencies`	`1, 2, 4, 8, 16, 32`	Concurrency levels executed sequentially in the automated load-test loop.
`cooldown`	`15`	Cooldown delay in seconds between consecutive concurrency runs.
`audio_dir`	`testfiles`	Directory containing WAV audio files.
`chunk_ms`	`250`	Audio frame chunk size in milliseconds.
`stagger_ms`	`50.0`	Startup stagger delay between workers in milliseconds.
`pace`	`1.0`	Pacing multiplier (`1.0` = real-time human speed, `0.0` = burst rate).
`gateway_http`	`http://192.168.112.9:3000`	API Gateway REST base URL.
`gateway_ws`	`ws://192.168.112.9:3000`	API Gateway WebSocket streaming URL.
`agent`	`whisper-v3-turbo`	Whisper STT agent/model used for transcription.
`tenant`	`ARCAAI`	Tenant identifier used for authentication.
`user`	`nitik`	Gateway authentication username.
`password`	`securepassword123`	Gateway authentication password.
`remote_diagnostics`	`true`	Enables remote GPU monitoring and diagnostic log collection.
`gpu_host`	`192.168.112.2`	Remote GPU server host address.

`gpu_user`	`nitik`	SSH username for the remote GPU server.
`gpu_sample_ms`	`50`	Remote `nvidia-smi` sampling interval in milliseconds.
`clear_redis`	`true`	Clears Redis streaming state before starting the load test.
`redis_url`	`redis://192.168.112.9:6379`	Redis server connection URL.

5. Generated Output Artifacts

For each run at concurrency level N, artifacts are saved under `logs/concurrency_<N>/`:

File	Description
`vllm_test_output_<N>.txt`	Formatted summary table, pass/fail counts, 10-stage latency breakdown, and automated bottleneck diagnostic.
`vllm_load_test_<N>.csv`	Frame-by-frame record of every transcript received, including timestamps, RTT, and server inference time.
`vllm_session_audit_<N>.csv`	Audit proof mapping every worker to its Session ID, WebSocket URL, ticket, audio file, and packet count.
`gpu_<N>.csv`	Continuous time-series GPU telemetry recorded at 50 ms intervals (utilization %, memory used, power draw).
`nvidia_smi_<N>.txt`	Point-in-time snapshot of GPU status at the conclusion of the test.
`vllm_whisper_<N>.log`	Container execution and runtime logs from the `vllm-whisper` container on the GPU server.